

循环神经网络

数据科学与大数据技术专业

第 6 讲

1 导读

除本导读外, 本讲共有 5 个主体节. 它们从全连接网络处理序列的局限出发, 引入 RNN 的隐藏状态和时间展开, 再讨论序列任务形式、BPTT 与长程依赖困难, 最后讲 LSTM、GRU 以及现代循环式序列模型.

1. **全连接有什么问题:** 说明固定输入输出映射为什么难以处理带顺序、上下文和历史状态的序列问题.
2. **循环神经网络简介:** 说明 RNN 如何用隐藏状态保存历史信息, 并在时间维度上复用同一套参数.
3. **应用到机器学习:** 说明序列到类别、同步序列标注和异步序列到序列任务如何用 RNN 表达.
4. **参数学习与长程依赖问题:** 说明 BPTT 如何计算循环网络梯度, 以及梯度消失和梯度爆炸为什么会限制长程记忆.
5. **LSTM、GRU 与其他改进结构:** 说明门控机制如何控制写入、保留、遗忘和输出, 并讨论现代 RNN 与长序列建模.

如果细节都忘了: 最应该记住的是, RNN 的核心是用一个随时间更新的隐藏状态压缩历史信息. 这个状态让模型能处理变长序列和上下文依赖, 但也带来梯度长链路、记忆容量和并行效率问题; LSTM、GRU、RWKV、Mamba 等改进都在围绕这些问题做折中.

问题思考:

- 为什么“能处理任意长度序列”和“能无损记住任意长历史”不是一回事?

2 全连接有什么问题

本节导读:

- **本节目的:** 说明普通全连接前馈网络为什么不自然适合序列数据.
- **为什么学:** 只有先看清固定映射和固定窗口的局限, 才能理解 RNN 为什么要引入隐藏状态和参数共享.
- **核心内容:** 序列任务中当前输出常依赖历史上下文, 而前馈网络默认只学习固定维度输入到输出的静态映射.

- **最小例子:** 在 “arrive Taipei” 和 “leave Taipei” 中, 同一个词 “Taipei” 的槽位标签不同; 若模型只看当前词向量, 就无法区分它是目的地还是出发地.
- **最该记住:** 序列建模需要内部状态来承接历史, 不能只靠把当前输入送进一个固定函数.

前馈神经网络在固定维度的输入输出映射中非常有效, 但序列问题的核心并不只是“把一个向量映射成另一个向量”. 序列样本带有顺序、上下文和历史状态, 当前输出常常不仅由当前输入决定, 还由过去发生过什么决定. 循环神经网络正是为了解决这一类状态依赖问题而引入的.

1. 固定映射不能区分上下文

- 前馈网络通常学习一个静态函数

$$\hat{y} = F(\mathbf{x}; \theta).$$

对同一个输入 $\mathbf{x}$, 在参数 θ 固定时输出必然相同. 这正是它作为函数逼近器的优点, 也是它处理序列上下文时的局限.

- 以智能订票系统中的 slot filling 为例, “arrive Taipei on November 2” 与 “leave Taipei on November 2” 中都出现了同一个词 “Taipei”. 若把每个词独立输入前馈网络, 模型看到的 “Taipei” 向量完全相同, 就无法判断它是目的地还是出发地. 正确判断依赖前面的 “arrive” 或 “leave”, 因而需要模型保存并利用历史信息.
- 这里容易产生一个误解: 只要把前后几个词拼接成更长输入, 前馈网络似乎也能利用上下文. 这只能处理固定长度窗口内的信息, 并且窗口一旦变化就需要重新设计输入表示; 它没有真正的内部状态, 因而不能自然地处理任意长序列.

2. 固定输入输出尺寸限制了序列建模

- 全连接前馈网络要求输入维度和输出维度预先固定. 但序列任务常常包含变长输入或变长输出, 例如输入一张图片生成一句描述是一对多问题, 输入一段视频判断动作类别是多对一问题, 机器翻译和逐帧标注则是多对多问题.
- 对变长序列做截断或补零可以满足工程上的张量形状要求, 但这只是把数据强行整理成固定维度, 并没有回答“历史如何影响当前决策”这一建模问题. 若直接把整段序列展平成一个大向量, 参数量还会随最大长度增长, 并且难以共享不同时间位置上的规律.
- 因此, 序列模型需要两类能力: 一是同一套参数可以在不同时间步重复使用, 二是模型内部需要有状态来承接过去的信息. 这两点都不是普通全连接前馈网络的默认结构.

3. 计算模型视角下的状态记忆

- 有限状态自动机由有限个状态、输入符号和状态转移规则构成. 读入一个符号后, 自动机根据当前状态和当前输入更新到下一个状态, 可写成

$$s_t = \delta(s_{t-1}, x_t).$$

这里的 s_{t-1} 就是对历史的压缩记忆.

- 图灵机进一步包含控制状态、读写头和可读写的纸带. 它之所以能模拟所有可计算过程, 是因为纸带提供了可扩展的存储, 状态转移规则则描述了逐步执行的程序. 需要注意, 这是一种关于计算过程的模型, 不是一次性的输入输出函数.

- 前馈网络的通用近似定理说明, 在固定维度和适当条件下, 它可以逼近连续函数. 但这并不等于它天然可以模拟任意长的程序执行过程. 若不把历史显式放进输入, 前馈网络没有类似 s_t 的可更新内部状态; 若把所有历史都放进输入, 又回到了固定窗口和固定长度的问题.

4. 动力系统与时滞现象

- 在数学建模中, 当前变化率依赖过去状态是很常见的. 例如带时滞的种群增长模型可写为

$$\frac{dx(t)}{dt} = rx(t) \left(1 - \frac{x(t-\tau)}{K} \right),$$

其中 $x(t)$ 表示当前种群数量, $x(t-\tau)$ 表示 τ 时间之前的种群数量, K 为环境容量. 这个式子表达的是: 当前增长不仅由当前规模决定, 还受到过去资源消耗和繁殖周期的影响.

- 类似地, 城市人口、工业生产线、机械制动与扭矩控制系统中, 当前输出常常受到历史观测和历史控制量影响. 若模型只看当前输入, 就无法刻画这类延迟效应.
- 从这个角度看, 序列神经网络并不是凭空出现的技巧, 而是把“状态随时间演化”的思想引入可学习模型.

5. 延时神经网络只能提供有限窗口记忆

- 延时神经网络 (Time Delay Neural Network, TDNN) 把过去若干时刻的输入或上一层活性值作为当前输入的一部分. 若第 l 层在时间 t 的活性值记为 $\mathbf{a}_t^{(l)}$, 一个典型形式为

$$\mathbf{a}_t^{(l)} = f \left(\mathbf{b}^{(l)} + \sum_{k=0}^K \mathbf{W}_k^{(l)} \mathbf{a}_{t-k}^{(l-1)} \right).$$

- 这个结构与一维时间卷积非常接近: 同一组权重在时间维度上滑动使用, 因而可以降低参数数量并利用局部时序模式. 若只看一层线性组合, 它本质上就是对时间窗口内输入做卷积式聚合.
- 需要注意, TDNN 的当前层并没有使用自身上一时刻的活性值 $\mathbf{a}_{t-1}^{(l)}$ 作为反馈状态. 因此它的“记忆”主要来自长度为 $K+1$ 的固定窗口, 窗口之外的信息不会直接影响当前输出. 这也是它与真正循环结构的关键区别.

6. 自回归与非线性自回归模型

- 自回归模型 (Autoregressive Model, AR) 用变量自身的历史值预测当前值, 基本形式为

$$\mathbf{y}_t = \mathbf{w}_0 + \sum_{k=1}^K \mathbf{W}_k \mathbf{y}_{t-k} + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(\mathbf{0}, \Sigma).$$

若没有外部输入, 它主要描述的是系统输出自身的动力学演化.

- 在控制和系统辨识中, 常常还需要把外部观测、控制信号、制动量或扭矩等变量纳入模型. 有外部输入的非线性自回归模型可写为

$$\mathbf{y}_t = f \left(\mathbf{x}_t, \mathbf{x}_{t-1}, \dots, \mathbf{x}_{t-K_x}, \mathbf{y}_{t-1}, \mathbf{y}_{t-2}, \dots, \mathbf{y}_{t-K_y} \right),$$

其中 $f(\cdot)$ 可以由前馈网络近似, K_x 和 K_y 是输入与输出历史窗口的长度.

- 这类模型比 TDNN 更一般, 因为当前输出显式依赖过去输出, 而过去输出又间接包含更早的输入信息. 但它仍然需要人为指定有限阶数 K_x, K_y , 并且递推误差可能沿时间累积. RNN 的做法是把这类历史依赖压缩到可学习的隐藏状态中, 用统一的状态更新公式处理序列.

问题思考:

- 前馈网络可以逼近固定维度函数, 但为什么这并不等价于它可以自然处理任意长序列?
- TDNN、AR/NARX 与 RNN 的记忆方式分别依赖什么对象?
- 在 slot filling 例子中, 为什么同一个词 “Taipei” 需要根据历史上下文输出不同标签?

3 循环神经网络简介

本节导读:

- **本节目的:** 引入 RNN 的隐藏状态、递推公式、时间展开和参数共享.
- **为什么学:** RNN 是理解序列模型、语言模型、BPTT、长程依赖和门控结构的基础.
- **核心内容:** RNN 在每个时间步用当前输入和上一隐藏状态计算新隐藏状态, 从而把历史压缩进固定维度向量.
- **最小例子:** 判断一个二值序列中当前位置是否形成连续两个 1, 只需隐藏状态保存上一时刻输入, 当前输入再与它共同决定输出.
- **最该记住:** RNN 的记忆不是保存全部历史, 而是用可学习递推把对任务有用的历史压缩进隐藏状态.

循环神经网络 (Recurrent Neural Network, RNN) 在网络中引入反馈连接, 使当前计算不只依赖当前输入, 还依赖上一时刻留下的内部状态. 从直观角度看, 它是一个带有记忆的神经网络; 从计算图角度看, 它是在时间维度上反复复用同一套参数的递推模型.

1. 记忆区块与隐藏状态

- 在普通前馈网络中, 隐藏层活性值在一次前向传播结束后就被用于产生输出, 并不会自然保留到下一次输入. RNN 则把上一个时间步的隐藏状态保存下来, 下一步再与新的输入共同参与计算. 这可以直观理解为网络内部有一个 memory block, 其中存放的是历史信息的压缩表示.
- 需要注意, 这里的记忆不是把所有历史样本原样存储起来. 若隐藏状态记为 $\mathbf{h}_t$, 则它通常是一个固定维度向量, 只能以压缩形式携带过去的信息. 因此, RNN 有能力利用历史, 但并不保证能无损记住任意长历史.
- 序列顺序会直接影响隐藏状态的演化. 即使一组词完全相同, 只要输入顺序不同, 递推路径就不同, 最终状态和输出也可能不同. 这正是 RNN 与 bag-of-words 式表示的重要区别.

2. 基本状态更新公式

- 给定输入序列 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$, RNN 在每个时间步维护隐藏状态 $\mathbf{h}_t$. 最常见的简单 RNN 可写为

$$\mathbf{z}_t = \mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b},$$

$$\mathbf{h}_t = f(\mathbf{z}_t),$$

$$\hat{\mathbf{y}}_t = g(\mathbf{V}\mathbf{h}_t + \mathbf{c}).$$

其中 $\mathbf{W}$ 连接输入到隐藏层, $\mathbf{U}$ 连接上一时刻隐藏状态到当前隐藏层, $\mathbf{V}$ 连接隐藏层到输出层. 在 vanilla RNN 中, f 常取 $\tanh$, 因为其输出范围为 $(-1, 1)$, 比 Sigmoid 更零中心化, 也能在反复递推中限制状态数值范围.

- 与前馈网络相比, 关键新增项是 $\mathbf{U}\mathbf{h}_{t-1}$. 它使得当前活性值不仅取决于当前输入 $\mathbf{x}_t$, 也取决于过去压缩形成的状态 $\mathbf{h}_{t-1}$. 因此, 即使当前输入相同, 只要历史不同, $\mathbf{h}_t$ 和输出就可能不同.
- 隐藏状态更新和输出计算是两个相关但不同的步骤. $\mathbf{U}$ 与 $\mathbf{W}$ 决定怎样更新内部状态, $\mathbf{V}$ 和输出函数 g 决定怎样把当前状态转成任务需要的预测. 例如分类任务中 g 可以包含 Softmax, 回归或序列生成任务中则可以换成其他输出层.

3. 时间展开与参数共享

- 若把 RNN 沿时间展开, 可以得到一个很深的前馈计算图:

$$\mathbf{h}_1 \rightarrow \mathbf{h}_2 \rightarrow \dots \rightarrow \mathbf{h}_T.$$

画成展开图时, 每个时间步看起来像一个新的网络层, 但它们并不是互不相同的网络. 所有时间步共用同一个状态更新函数和同一套参数 $\mathbf{U}, \mathbf{W}, \mathbf{b}$.

- 参数共享有两个作用. 一方面, 模型可以处理不同长度的序列, 参数量不会随 T 线性增长; 另一方面, 同一类时序规律可以在任意时间位置被识别. 这与卷积网络在空间位置上共享卷积核的思想相似, 只是 RNN 的共享发生在时间维度上.
- 初始状态 $\mathbf{h}_0$ 通常设为零向量, 也可以设为可学习参数或由另一网络根据上下文生成. 这里容易混淆的是, $\mathbf{h}_0$ 不是输入序列的一部分, 而是递推计算开始前给内部状态的初始条件.

4. RNN 的记忆方式

- 展开递推式可知, $\mathbf{h}_t$ 是 $\mathbf{x}_1, \dots, \mathbf{x}_t$ 的函数, 即

$$\mathbf{h}_t = F_t(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t).$$

这就是 RNN 的记忆: 它并不把所有历史样本原样存下来, 而是把历史信息压缩到固定维度的隐藏状态中.

- 与 TDNN 相比, RNN 的状态具有自反馈, 不再局限于显式窗口 K 内的输入. 与 AR/NARX 相比, RNN 不必把有限个历史输出逐项列入输入, 而是用隐藏状态学习应该保留哪些信息.
- 需要注意, 理论上可以处理任意长度序列并不表示它能无损记住任意长历史. 隐藏状态维度有限, 训练时还会遇到梯度消失或梯度爆炸, 因而标准 RNN 在长程依赖上仍然存在实际困难.

5. 与生物反馈、联想记忆和理论表达能力的关系

- 生物神经回路中存在大量反馈连接, 当前神经活动会影响后续活动状态. RNN 借用了这种“状态反馈”的思想, 因而比纯前馈网络更接近某些生物神经系统的结构特征. 需要注意, 这里的类比主要发生在结构层面: RNN 不是完整的生物神经系统模型, 而是把反馈连接抽象成可学习的状态递推.
- 神经网络除了作为输入输出映射的机器学习模型外, 也可以作为记忆存储与检索模型. Hebb 学习思想常被概括为: 经常共同激活的神经元连接会增强, 不共同激活的连接会减弱. 这说明记忆可以编码在连接强度中, 而不一定存放在某个单独位置.
- RNN 中的“记忆”还包含另一层含义: 历史信息会被不断写入隐藏状态, 随后又通过同一套递推规则影响未来输出. 因此, 它既不同于查表式记忆, 也不同于把全部历史样本保存在外部存储中. 它更像是一种动态记忆, 其内容会随输入序列持续更新.
- 典型的联想记忆模型如 Hopfield 网络, 可以通过网络状态收敛来恢复存储模式. 后续学习注意力机制时还会再次看到类似思想: 查询向量根据相似性从一组记忆表示中读取信息. 二者机制不同, 但都体现了“表示、存储、检索”之间的联系.
- 从动力系统角度看, 完全连接的循环网络可以在适当条件下近似一类非线性动态系统. 直观地说, 它不仅学习某个静态映射, 还学习状态如何随输入和时间演化. 这也是 RNN 适合处理语音、文本、时间序列和控制信号的重要原因.
- 从计算理论角度看, 图灵完备指一种计算机制能够模拟任意图灵机, 因而原则上可以执行所有可计算过程. Siegelmann 和 Sontag 的相关结果表明, 在理想化条件下, 由 Sigmoid 型神经元构成的循环网络可以模拟图灵机. 这说明循环结构在表达能力上非常强, 不只是有限窗口的模式匹配器.
- 这个结论应理解为表达能力的理论边界, 而不是训练保证. 它依赖理想化的实数精度、足够时间和合适参数; 实际机器学习中, 数据、优化、数值精度和泛化能力仍然决定模型是否可用. 因此, 不能因为 RNN 具有很强的理论表达能力, 就忽略梯度传播、记忆容量和训练稳定性等现实问题.

问题思考:

- RNN 的隐藏状态与人类记忆有哪些相似处和差异?
- 为什么 RNN 的图灵完备性不能直接推出它在实际任务中一定容易训练?
- 时间展开后, RNN 与普通深层前馈网络在参数共享和梯度传播上有什么不同?

3.1 手动实现一个 RNN 的例子

为了说明 RNN 的隐藏状态如何保存历史信息, 这里手动构造一个最小例子: 给定一个二值序列, 网络在每个时间步判断当前位置是否与前一位置共同形成连续的两个 1.

1. 任务目标

输入是一个二值序列

$$X = (x_1, x_2, \dots, x_T), \quad x_t \in \{0, 1\}.$$

输出也是一个按时间步给出的二值序列

$$Y = (y_1, y_2, \dots, y_T), \quad y_t \in \{0, 1\},$$

其中

$$y_t = \begin{cases} 1, & x_{t-1} = 1 \text{ 且 } x_t = 1, \\ 0, & \text{otherwise.} \end{cases}$$

为了处理第一个时间步, 可约定序列开始前 $x_0 = 0$. 这样 $y_t = 1$ 的含义是: 在时间 t 结束时刚看到一个相邻模式 11. 需要注意, 这不是判断“一段恰好长度为 2 的 1”; 如果输入中出现 111, 那么第二个和第三个 1 都会触发一次检测.

2. 从输出公式反推隐藏状态

先不急着写循环矩阵, 而是从希望实现的输出层开始. 对二值变量 $x_t, x_{t-1} \in \{0, 1\}$, 连续两个 1 可以由一个 ReLU 阈值函数表示:

$$y_t = \text{ReLU}(x_t + x_{t-1} - 1).$$

因为四种输入组合对应的括号内数值分别为 $-1, 0, 0, 1$, ReLU 后只有 $(x_{t-1}, x_t) = (1, 1)$ 得到 1.

若输出层写成 RNN 的标准形式

$$y_t = \text{ReLU}(\mathbf{W}_{hy}\mathbf{h}_t) = \text{ReLU}(x_t + x_{t-1} - 1),$$

那么隐藏状态 $\mathbf{h}_t$ 至少要让输出层读到三项信息: 当前输入 x_t , 上一时刻输入 x_{t-1} , 以及一个常数 1 用来产生 -1 这个偏置. 因此最直接的选择是

$$\mathbf{h}_t = \begin{bmatrix} x_t \\ x_{t-1} \\ 1 \end{bmatrix}, \quad \mathbf{W}_{hy} = \begin{bmatrix} 1 & 1 & -1 \end{bmatrix}.$$

这里第 1 个隐藏单元保存当前输入, 第 2 个隐藏单元保存上一步输入, 第 3 个隐藏单元始终为 1. 这一维常数状态相当于把输出层的偏置合并进矩阵乘法.

3. 反推循环更新矩阵

现在要求这个隐藏状态确实能由一个 RNN 递推得到. 使用如下简单结构:

$$\begin{aligned} \mathbf{h}_t &= \text{ReLU}(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}x_t), \\ y_t &= \text{ReLU}(\mathbf{W}_{hy}\mathbf{h}_t), \end{aligned}$$

并令初始状态为

$$\mathbf{h}_0 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix},$$

这与 $x_0 = 0$ 的约定一致.

假设上一时刻已经有

$$\mathbf{h}_{t-1} = \begin{bmatrix} x_{t-1} \\ x_{t-2} \\ 1 \end{bmatrix}.$$

当前时刻希望得到

$$\mathbf{h}_t = \begin{bmatrix} x_t \\ x_{t-1} \\ 1 \end{bmatrix}.$$

也就是说, 第 1 维应当从当前输入 x_t 写入, 第 2 维应当从上一状态的第 1 维复制过来, 第 3 维应当从上一状态的第 3 维继续保持为 1. 按这三个要求逐行反推矩阵:

- 第 1 行要产生 x_t , 因而从输入到隐藏的权重取 1, 从旧状态来的权重全取 0.
- 第 2 行要产生 x_{t-1} , 它正好是 $\mathbf{h}_{t-1}$ 的第 1 个分量, 因而循环矩阵第 2 行取 $[1, 0, 0]$.
- 第 3 行要保持常数 1, 因而循环矩阵第 3 行取 $[0, 0, 1]$, 并且不接收当前输入.

于是得到

$$\mathbf{W}_{xh} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{W}_{hh} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{W}_{hy} = \begin{bmatrix} 1 & 1 & -1 \end{bmatrix}.$$

代回检查:

$$\mathbf{W}_{hh}\mathbf{h}_{t-1} = \begin{bmatrix} 0 \\ x_{t-1} \\ 1 \end{bmatrix}, \quad \mathbf{W}_{xh}x_t = \begin{bmatrix} x_t \\ 0 \\ 0 \end{bmatrix}.$$

两者相加为

$$\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}x_t = \begin{bmatrix} x_t \\ x_{t-1} \\ 1 \end{bmatrix}.$$

由于这里的三个分量都非负, ReLU 不会改变它们. 因此由归纳可知, 该 RNN 的隐藏状态始终满足 $\mathbf{h}_t^\top = (x_t, x_{t-1}, 1)$.

4. 例子

以输入序列

$$X = (0, 1, 0, 1, 1, 1, 0, 1, 1)$$

为例, 逐步递推如下:

t	x_t	$\mathbf{h}_t^\top = (x_t, x_{t-1}, 1)$	y_t
1	0	(0, 0, 1)	0
2	1	(1, 0, 1)	0
3	0	(0, 1, 1)	0
4	1	(1, 0, 1)	0
5	1	(1, 1, 1)	1
6	1	(1, 1, 1)	1
7	0	(0, 1, 1)	0
8	1	(1, 0, 1)	0
9	1	(1, 1, 1)	1

因此输出为

$$Y = (0, 0, 0, 0, 1, 1, 0, 0, 1).$$

这个例子展示了 RNN 的一个关键思想：隐藏状态不必保存完整历史，只要保存解决当前任务所需的充分信息即可。对“连续两个 1”这个任务来说，当前输入和上一时刻输入就是充分信息；矩阵 $\mathbf{W}_{hh}$ 的作用正是把上一时刻的当前输入向后移动一格，形成可被输出层读取的一步记忆。

4 应用到机器学习

本节导读：

- **本节目的：**说明 RNN 在机器学习任务中的几种常见输入输出模式。
- **为什么学：**只有明确输入序列和输出序列的时间关系，才能选择合适的汇聚、标注或解码方式。
- **核心内容：**序列任务可分为序列到类别、同步序列到序列、异步序列到序列和 Encoder-Decoder 等形式。
- **最小例子：**情感分类把整句话读完后输出一个标签，中文分词则要求每个字输出一个位置标签，机器翻译还需要生成长度可能不同的目标序列。
- **最该记住：**RNN 提供的是状态递推框架，具体任务还要决定在哪些时间步读入、汇聚和输出。

RNN 的输入和输出都可以放在时间轴上理解。不同任务的区别主要在于：模型需要在一个时间步输出结果，还是需要在每个时间步输出结果；输入序列和输出序列的长度是否一致。

1. 序列到类别模式

- 序列到类别模式把整个输入序列映射成一个标签，例如给一句话判断情绪，给一段传感器信号判断设备状态，或给一段用户行为序列判断是否异常。此时可以先用 RNN 逐步读入 $\mathbf{x}_1, \dots, \mathbf{x}_T$ ，再用最终隐藏状态完成分类：

$$\hat{\mathbf{y}} = G(\mathbf{h}_T).$$

其中 G 可以是简单线性分类器，也可以是多层前馈网络。

- $\mathbf{h}_T$ 可以看作模型读完整个序列后的摘要。但当序列很长时，最早的信息可能已经在递推中被弱化，因而也常用平均池化、最大池化或注意力加权等方式从所有隐藏状态 $\mathbf{h}_1, \dots, \mathbf{h}_T$

中构造序列表示。需要注意，平均不是为了满足公式形式，而是为了让多个时间位置都能直接参与最终判断。

- 文本情感分类通常先把离散词或字转换为向量。最常见做法是查表式 word embedding：每个 token 对应一个可学习向量，训练时这些向量会与 RNN 参数一起更新。因此，词嵌入不是人工固定的编码，而是模型参数的一部分。

2. 同步的序列到序列模式

- 同步的序列到序列模式指输入和输出在时间位置上一一对应，即每个输入位置都有一个输出标签：

$$\hat{y}_t = G(h_t), \quad t = 1, \dots, T.$$

这类任务又称序列标注，常见于中文分词、词性标注、命名实体识别和逐帧动作识别。

- 中文分词是自然语言处理中的基础任务，因为中文书写中词语之间没有天然空格。例如一个字符串可以用 B、M、E、S 标注每个字在词中的位置，分别表示 begin、middle、end、single。RNN 读入字符 embedding 后，在每个位置输出相应标签。
- 这里容易误以为同步任务总是简单的逐点分类。实际上，当前字是否成词常取决于左右上下文。例如某些短语中的边界并不只由当前字决定，因而隐藏状态中的历史信息仍然重要；双向 RNN 还会进一步利用未来上下文。

3. 异步的序列到序列模式

- 异步序列到序列任务中，输入长度和输出长度不必相同。机器翻译、语音识别和文本摘要都属于这一类。例如中文翻英文和英文翻中文都不存在一方一定更长或更短的规律，输出长度由语义、语言结构和表达方式共同决定。
- 这类任务通常不能简单地要求第 t 个输入对应第 t 个输出。语音识别中，多个声学帧可能对应一个字或一个音素；机器翻译中，一个短语可能整体移动位置。因此，模型需要先理解整段输入，再逐步产生输出。
- 信息抽取 (Information Extraction, IE) 介于序列标注和结构化预测之间。它的目标是从无结构文本中抽取实体、关系或事件，形成结构化信息。若输出是每个 token 的标签，可以用同步序列标注；若输出是若干三元组或事件结构，则需要更复杂的解码过程。

4. Encoder-Decoder 架构

- Encoder-Decoder 架构是处理异步序列任务的经典思路。编码器读取输入序列并形成上下文表示，解码器再根据该表示逐步生成输出：

$$\begin{aligned} h_t^{\text{enc}} &= F_{\text{enc}}(x_t, h_{t-1}^{\text{enc}}), \\ s_m^{\text{dec}} &= F_{\text{dec}}(y_{m-1}, s_{m-1}^{\text{dec}}, c). \end{aligned}$$

其中 c 是编码器提供给解码器的上下文信息。

- 早期 Encoder-Decoder 常把整段输入压缩成一个固定长度向量，这在长句上会形成信息瓶颈。后来的注意力机制让解码器在每一步从所有编码器状态中读取信息，正是为了解决单一上下文向量容量有限的问题。

- 从应用角度看, RNN 的核心价值不是某一种固定输出形式, 而是提供了一个统一的状态递推框架. 只要明确输入和输出在时间轴上的关系, 就可以设计相应的读入、汇聚和解码方式.

问题思考:

- 序列到类别任务中, 使用 $\mathbf{h}_T$ 、平均池化和注意力汇聚分别有什么优缺点?
- 为什么中文分词虽然是同步序列标注, 仍然需要上下文信息?
- 机器翻译中, 为什么不能假设输入序列和输出序列必然等长?

4.1 字符级语言模型

字符级语言模型 (character-level language model) 是理解 RNN 如何生成文本的一个很好的入口. Karpathy 在博客文章 *The Unreasonable Effectiveness of Recurrent Neural Networks* 中用一系列实验展示了这种模型的效果: 只让模型预测下一个字符, 它就能逐渐学会英文拼写、换行格式、Wiki 标记、LaTeX 结构, 甚至生成看起来像 Linux C 源码的缩进和括号模式¹.

这个例子有趣之处在于, 任务本身极其简单, 但序列状态把许多局部预测串成了复杂行为.

1. 下一个字符预测就是逐时间步分类

- 设字符表为 $\mathcal{V}$, 大小为 K , 一段文本为

$$c_1, c_2, \dots, c_T, \quad c_t \in \mathcal{V}.$$

字符级语言模型在时间 t 看到前缀 $c_1, \dots, c_t$, 目标是预测下一个字符 c_{t+1} . 因此训练样本不需要人工标注: 把原始文本右移一位, 就自动得到每个时间步的监督信号.

- 一个基本 RNN 语言模型可写为

$$\begin{aligned} \mathbf{e}_t &= E[c_t], \\ \mathbf{h}_t &= \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{e}_t + \mathbf{b}_h), \\ \mathbf{z}_t &= \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \\ p_\theta(c_{t+1} = v \mid c_{\leq t}) &= \text{softmax}(\mathbf{z}_t)_v, \quad v \in \mathcal{V}. \end{aligned}$$

这里 $\mathbf{z}_t$ 是 logit, 不是概率; Softmax 之后才得到字符表上的概率分布. 这个区分很重要, 因为训练损失、采样温度和贪心解码都作用在这些得分与概率之间的关系上.

- Karpathy 博客中的 “hello” 例子直接说明了上下文的作用. 输入字符 “h” 时, 目标是 “e”; 输入前缀 “he” 时, 目标是 “l”; 输入 “hel” 时, 目标仍是 “l”; 输入 “hell” 时, 目标变成 “o”. 同一个输入字符 “l” 在不同上下文下对应不同目标, 因而模型不能只看当前字符, 必须依赖循环状态记录已经读到哪里.

2. 字符如何进入神经网络

- 离散字符不能直接参与矩阵乘法. 最朴素的表示是 one-hot 向量 $\mathbf{x}_t \in \{0, 1\}^K$, 其中只有字符 c_t 对应的位置为 1. 这正是博客中 1-of- K 编码的含义.

¹ Andrej Karpathy, *The Unreasonable Effectiveness of Recurrent Neural Networks*, 2015, <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>.

- 工程实现中常使用可学习 embedding 矩阵 $E \in \mathbb{R}^{K \times d}$. 若把 c_t 看作字符索引, 则

$$\mathbf{e}_t = E[c_t].$$

从线性代数角度看, 这等价于用 one-hot 向量选择矩阵的一行; 从实现角度看, 它是一次查表操作, 比显式构造稀疏 one-hot 再做大矩阵乘法更直接.

- embedding 也是模型参数. 它不是预先写死的“字符含义表”, 而是在预测下一个字符的损失下被反向传播不断调整. 对字符级模型而言, embedding 未必能学出像词向量那样清晰的语义邻近关系, 但它能提供一个低维、稠密、可优化的输入空间.

3. 训练损失: 每一步都要为下一个字符负责

- 对整段训练序列, 常用逐时间步交叉熵:

$$\mathcal{L} = - \sum_{t=1}^{T-1} \log p_{\theta}(c_{t+1} | c_{\leq t}).$$

如果模型在第 t 步把正确字符的概率给得太低, 这一项损失就会变大, 反向传播会同时调整输出层、循环权重和输入 embedding.

- 需要注意, “惩罚错误字符”并不是单独对某个错误选项打分, 而是通过 Softmax 与交叉熵整体改变概率分布. 对 one-hot 目标, logit 的梯度具有熟悉形式

$$\frac{\partial \mathcal{L}_t}{\partial z_{t,k}} = \hat{p}_{t,k} - y_{t,k}.$$

因此正确字符的 logit 会被推高, 概率过高的错误字符会被压低.

- 训练时通常把真实的前一个字符喂给模型, 这常称为 teacher forcing. 生成时则没有标准答案可用, 模型必须把自己刚生成的字符作为下一步输入. 这一区别解释了为什么训练损失很低的模型, 在长篇自由生成时仍可能逐渐偏离数据分布: 如果前面采样出不自然字符, 后续上下文也会被模型自己的输出改变.

4. 生成文本: 反馈、采样与温度

- 生成阶段从一个起始字符或短前缀开始. 模型计算 $p_{\theta}(c_{t+1} | c_{\leq t})$, 从中选出下一个字符 $\tilde{c}_{t+1}$, 再把它作为下一步输入. 如此反复, 就得到一段自回归生成文本.
- 最简单的选择是贪心解码:

$$\tilde{c}_{t+1} = \operatorname{argmax}_{v \in \mathcal{V}} p_{\theta}(v | c_{\leq t}).$$

它确定、稳定, 但给定同一前缀时输出永远相同, 在开放式文本中还容易进入重复循环. Karpathy 的实验中, 很低温度的采样就会让模型反复生成同一类短语, 这正说明“每一步都选最像训练集的局部答案”不等于“整段文本最自然”.

- 更常见的做法是按概率分布采样. 为了控制保守程度, 可以在 Softmax 前加入温度参数 $\tau > 0$:

$$p_i^{(\tau)} = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}.$$

当 $\tau < 1$ 时, 分布更尖锐, 模型更保守; 当 $\tau > 1$ 时, 低概率字符也更可能被选中, 文本更多样, 但拼写、括号和语法错误也会增加. Beam search 则保留若干候选前缀并比较序列

得分, 更适合有明确目标的翻译或摘要, 对开放式创作未必总是更有趣。

5. 为什么这个小模型会显得“会写东西”

- 字符级预测看似只学局部统计, 但 RNN 的隐藏状态会把历史压缩进 h_t . 当训练文本包含稳定结构时, 模型可以逐渐学到多层次规律: 先学常见字符组合和空格, 再学单词拼写, 再学换行、缩进、括号闭合和局部格式.
- 博客中的几个例子尤其能显示这种机制的表现. 在 Shakespeare 文本上, 模型会生成角色名、换行和近似戏剧台词; 在 Wikipedia 文本上, 它会生成类似链接、标题和 XML 标签的结构; 在 LaTeX 源码上, 它甚至能产生看起来接近可编译的数学排版; 在 Linux 源码上, 它会模仿注释、缩进、花括号和 'if' 语句. 这些现象说明, “预测下一个字符”这个目标可以迫使模型吸收大量关于形式结构的规律.
- 但也要避免把这种现象解释过头. 字符模型生成的 URL、定理或代码片段可能只是形式上像真的, 并不保证事实正确或程序可运行. 这与现代语言模型中的幻觉问题一脉相承: 语言模型首先学习的是条件分布, 不是外部世界的真实性验证器.

6. 从字符级 RNN 到现代大语言模型

- 现代大语言模型通常不再逐字符建模, 而是使用 token, 例如子词、词片段或字节片段. 训练目标也从

$$p(c_{t+1} | c_{\leq t})$$

平移为

$$p(x_{t+1} | x_{\leq t}),$$

其中 x_t 是 token. 因此, “给定上文预测下一个符号”这一自监督思想仍然相同.

- 真正变化的是架构和规模. 现代 LLM 多使用 Transformer, 依靠自注意力直接读取上下文中的多个位置, 而不是把所有历史压缩进一个循环隐藏状态. 这缓解了 RNN 长距离依赖中的信息瓶颈, 也让训练可以在时间维度上更充分并行.
- 因此, 字符级 RNN 不是现代 LLM 的缩小版, 但它是理解语言模型生成机制的最小清晰例子: 离散符号先变成向量, 模型给出下一个符号的概率分布, 训练时用右移文本提供监督, 生成时把自己的输出接回输入. 理解这条链路之后, tokenization、Transformer 和注意力机制的作用也会更自然地显现出来.

思考题:

- 为什么字符级语言模型不需要人工为每个位置单独打标签?
- 同一个字符在不同上下文下可能对应不同下一个字符, 这说明隐藏状态需要保存哪些信息?
- 在开放式文本生成中, 贪心解码、按分布采样和低温度采样分别会带来什么偏差?

5 参数学习与长程依赖问题

本节导读:

- 本节目的: 说明 RNN 参数如何通过时间展开做反向传播, 以及长程依赖为什么难学.

- **为什么学:** RNN 看似能携带任意长历史, 但训练时梯度要沿时间链路反传, 这会直接限制模型能学到多远的依赖.
- **核心内容:** BPTT 把所有时间步的共享参数梯度累加起来; 反向信号中的 Jacobian 连乘会导致梯度消失或爆炸.
- **最小例子:** 若每向前反传一步梯度都乘以约 0.5, 经过 20 步后只剩约 0.5^{20} , 早期输入几乎无法影响参数更新.
- **最该记住:** 长程依赖困难不是 RNN “理论上不能看远”, 而是梯度和状态在长时间链路上传递不稳定.

RNN 的参数学习可以理解为把网络沿时间展开后做反向传播. 与普通前馈网络不同的是, 同一组参数会在多个时间步重复出现, 因而梯度需要把所有时间步上的贡献累加起来.

1. 序列损失与时间展开

- 给定训练样本 $(\mathbf{x}, \mathbf{y})$, 其中

$$\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_T), \quad \mathbf{y} = (y_1, \dots, y_T),$$

对同步序列标注任务, 每个时间步都可以定义瞬时损失

$$\mathcal{L}_t = \ell(y_t, g(\mathbf{h}_t)), \quad \mathcal{L} = \sum_{t=1}^T \mathcal{L}_t.$$

若是序列到类别任务, 通常只在最后一个或少数几个输出位置定义损失; 若是异步生成任务, 损失则按输出时间步求和.

- 对简单 RNN, 前向递推为

$$\begin{aligned} \mathbf{z}_k &= \mathbf{U}\mathbf{h}_{k-1} + \mathbf{W}\mathbf{x}_k + \mathbf{b}, \\ \mathbf{h}_k &= f(\mathbf{z}_k), \quad 1 \leq k \leq t. \end{aligned}$$

随时间反向传播 (BackPropagation Through Time, BPTT) 的做法是先按时间展开计算图, 再从损失位置向前应用链式法则.

- 展开和折叠是两个不同视角. 展开是为了把循环依赖变成普通计算图上的路径; 折叠是因为这些时间步共享同一组参数, 最后必须把各时间步产生的梯度加到同一个参数矩阵上.

2. BPTT 的梯度形式

- 先考虑单个时间步损失 $\mathcal{L}_t$ 对循环权重 $\mathbf{U}$ 的贡献. 定义

$$\delta_{t,k} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{z}_k}, \quad 1 \leq k \leq t.$$

对矩阵元素 u_{ij} , 因为 $\mathbf{z}_k$ 中第 i 个分量包含 $u_{ij}[\mathbf{h}_{k-1}]_j$, 所以

$$\frac{\partial \mathcal{L}_t}{\partial u_{ij}} = \sum_{k=1}^t [\delta_{t,k}]_i [\mathbf{h}_{k-1}]_j.$$

写成矩阵形式就是

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{U}} = \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^\top.$$

- 如果总损失为 $\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t$, 则共享参数的总梯度为

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{U}} &= \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^\top, \\ \frac{\partial \mathcal{L}}{\partial \mathbf{W}} &= \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{x}_k^\top, \\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}} &= \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k}. \end{aligned}$$

这些式子仍是误差项与输入活性的外积形式, 区别在于时间维度上多了一层累加.

- 误差项本身也按时间递推. 对 $1 \leq k < t$, 有

$$\begin{aligned} \delta_{t,k} &= \frac{\partial \mathbf{h}_k}{\partial \mathbf{z}_k} \frac{\partial \mathbf{z}_{k+1}}{\partial \mathbf{h}_k} \frac{\partial \mathcal{L}_t}{\partial \mathbf{z}_{k+1}} \\ &= \text{diag}(f'(\mathbf{z}_k)) \mathbf{U}^\top \delta_{t,k+1}. \end{aligned}$$

这里的对角矩阵来自逐元素激活函数, $\mathbf{U}^\top$ 来自循环连接. 这一步与第 4 讲中前馈网络误差项递推完全一致, 只是层编号换成了时间编号.

3. 梯度消失与梯度爆炸

- 为了看清楚问题的来源, 先考虑最简单的情形: 损失只定义在最后一个隐藏状态上,

$$\mathcal{L} = \mathcal{L}(\mathbf{h}_T),$$

RNN 的状态递推写为

$$\begin{aligned} \mathbf{z}_t &= \mathbf{M}_1 \mathbf{h}_{t-1} + \mathbf{M}_2 \mathbf{x}_t + \mathbf{b}, \\ \mathbf{h}_t &= f(\mathbf{z}_t). \end{aligned}$$

其中 $\mathbf{M}_1$ 是循环权重, 即上一时刻隐藏状态到当前隐藏状态的连接. 训练时对它做梯度下降:

$$\mathbf{M}_1 \leftarrow \mathbf{M}_1 - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{M}_1}.$$

因此关键问题变成: 最后一时刻的损失如何一路反传到各个时间步上对同一个 $\mathbf{M}_1$ 的使用.

- 从最后一步开始. 记

$$\mathbf{g}_T = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_T}, \quad \delta_T = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_T} = \mathbf{g}_T \odot f'(\mathbf{z}_T).$$

因为 $\mathbf{z}_T = \mathbf{M}_1 \mathbf{h}_{T-1} + \mathbf{M}_2 \mathbf{x}_T + \mathbf{b}$, 所以最后一步对循环权重的梯度贡献是

$$\left. \frac{\partial \mathcal{L}}{\partial \mathbf{M}_1} \right|_T = \boldsymbol{\delta}_T \mathbf{h}_{T-1}^\top.$$

这仍然是前馈网络中熟悉的形式: 误差信号乘以上一层输入. 区别在于, 这里的“上一层输入”是上一时刻的隐藏状态.

- 再往前退一步看 $T-1$. 损失 $\mathcal{L}$ 不直接依赖 $\mathbf{h}_{T-1}$, 但它通过 $\mathbf{h}_{T-1} \rightarrow \mathbf{z}_T \rightarrow \mathbf{h}_T \rightarrow \mathcal{L}$ 间接依赖它. 因此

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{T-1}} = \mathbf{M}_1^\top \boldsymbol{\delta}_T.$$

再乘上本时刻激活函数的导数, 得到

$$\boldsymbol{\delta}_{T-1} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{T-1}} = (\mathbf{M}_1^\top \boldsymbol{\delta}_T) \odot f'(\mathbf{z}_{T-1}).$$

因而 $T-1$ 时刻对同一个 $\mathbf{M}_1$ 的梯度贡献为

$$\left. \frac{\partial \mathcal{L}}{\partial \mathbf{M}_1} \right|_{T-1} = \boldsymbol{\delta}_{T-1} \mathbf{h}_{T-2}^\top.$$

- 继续往前退到 $T-2$, 同样有

$$\begin{aligned} \boldsymbol{\delta}_{T-2} &= (\mathbf{M}_1^\top \boldsymbol{\delta}_{T-1}) \odot f'(\mathbf{z}_{T-2}) \\ &= \text{diag}(f'(\mathbf{z}_{T-2})) \mathbf{M}_1^\top \text{diag}(f'(\mathbf{z}_{T-1})) \mathbf{M}_1^\top \boldsymbol{\delta}_T. \end{aligned}$$

这一步已经显示出长程依赖的核心结构: 每向前多传一个时间步, 反向信号就要多乘一次 $\mathbf{M}_1^\top$ 和一次激活函数导数.

- 归纳可得, 对任意 $k < T$,

$$\boldsymbol{\delta}_k = \text{diag}(f'(\mathbf{z}_k)) \mathbf{M}_1^\top \boldsymbol{\delta}_{k+1}.$$

因为同一个循环权重 $\mathbf{M}_1$ 在所有时间步共享, 所以总梯度要把各时刻的贡献加起来:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{M}_1} = \sum_{k=1}^T \boldsymbol{\delta}_k \mathbf{h}_{k-1}^\top.$$

展开其中较早时间步的误差信号, 可以看到

$$\boldsymbol{\delta}_k = \text{diag}(f'(\mathbf{z}_k)) \mathbf{M}_1^\top \text{diag}(f'(\mathbf{z}_{k+1})) \mathbf{M}_1^\top \cdots \text{diag}(f'(\mathbf{z}_{T-1})) \mathbf{M}_1^\top \boldsymbol{\delta}_T.$$

因而从最终损失反传到很早的时间步时, 梯度不可避免地包含一长串 Jacobian 矩阵连乘.

- 梯度消失和梯度爆炸正是由这串连乘造成的. 若粗略记

$$\gamma_\tau = \|\text{diag}(f'(\mathbf{z}_\tau)) \mathbf{M}_1^\top\|,$$

则有近似关系

$$\|\boldsymbol{\delta}_k\| \lesssim \left(\prod_{\tau=k}^{T-1} \gamma_\tau \right) \|\boldsymbol{\delta}_T\|.$$

如果这些因子多数小于 1, 早期时间步传来的梯度会随距离指数衰减, 这就是梯度消失. Sigmoid 和 Tanh 在饱和区导数很小, 会加重这种现象; ReLU 虽然在正半轴导数为 1, 但关闭的单元导数为 0, 仍可能截断梯度路径. 反过来, 如果循环矩阵在某些方向上的放大作用持续大于 1, 这串连乘就可能让梯度指数增长, 形成梯度爆炸.

- 这里的梯度消失并不是说总梯度 $\partial\mathcal{L}/\partial\mathbf{M}_1$ 一定变成零. 总梯度是许多时间步贡献的和, 靠近 T 的项仍可能很大. 真正的问题是较早输入经过长链路传来的贡献被压得很小, 于是参数更新主要由短期依赖决定, 模型难以学习“很久以前的信息影响当前输出”这类规律.
- 梯度爆炸则会造成 $\mathbf{M}_1$ 的更新过大, 使损失震荡甚至出现数值溢出. 常用工程处理包括梯度截断

$$\mathbf{g} \leftarrow \frac{\tau}{\max(\|\mathbf{g}\|, \tau)} \mathbf{g},$$

以及适度的权重衰减、正交初始化或门控结构. 需要注意, 梯度截断主要处理爆炸导致的数值不稳定; 它不能让已经消失的长程梯度重新变大. 这也是后面引入 LSTM、GRU 等门控循环网络的重要动机.

问题思考:

- 为什么 BPTT 中共享参数的梯度需要累加所有时间步的贡献?
- 梯度消失时, 模型最先失去的是短期依赖还是长程依赖? 为什么?
- 梯度截断能缓解梯度爆炸, 为什么不能直接解决梯度消失?

6 LSTM、GRU 与其他改进结构

本节导读:

- **本节目的:** 说明门控循环网络如何缓解简单 RNN 的长程依赖和记忆控制问题.
- **为什么学:** 简单 RNN 的状态会不断被重写, 梯度也不稳定; 门控机制提供了可学习的信息保留、写入和遗忘方式.
- **核心内容:** GRU 用更新门和重置门控制隐藏状态, LSTM 用输入门、遗忘门、输出门和细胞状态区分存储与输出, 现代 RNN 则进一步追求长序列效率.
- **最小例子:** 若一句话开头出现否定词, LSTM 的遗忘门可以让相关信息在细胞状态中保留多个时间步, 直到后面情感判断需要它.
- **最该记住:** 门控的核心不是“记得越多越好”, 而是让模型学会什么该写入、什么该保留、什么该丢掉.

简单 RNN 的隐藏状态能够把历史压缩进当前表示, 但它同时面临三类实际困难: 远距离梯度难以稳定传播, 固定维度状态会形成记忆容量瓶颈, 以及时间步之间的递推依赖限制并行计算. 门控循环网络的目标不是取消状态记忆, 而是让模型学会控制信息何时写入、保留、读取和遗忘.

1. 线性捷径、残差视角与门控

- 一种直观改进是让隐藏状态包含线性捷径:

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t; \theta).$$

此时 $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1} = I$, 反向传播至少存在一条梯度为 1 的路径. 如果把时间步看成网络深度, 这种结构与残差网络中的恒等连接非常相似.

- 但状态更新不能只靠无条件累加. 更一般的形式是

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta).$$

这保留了状态之间的非线性作用, 但 Jacobian 变为 $I + \partial g / \partial \mathbf{h}_{t-1}$. 因此, 线性捷径可以缓解梯度消失, 但当非线性项的特征值过大时仍可能造成梯度爆炸.

- 记忆容量问题也不能只靠加法解决. 若模型持续写入而很少删除, 隐藏状态会逐渐混入过多无关信息; 若候选更新长期落在 Sigmoid 或 Tanh 的饱和区, 新旧信息又会变得难以区分. 因此, 长程记忆需要的不只是保留通道, 还需要可学习的遗忘机制.
- 门控机制 (gate) 用一个取值在 $[0, 1]$ 的向量控制信息流. 以简化形式看, 门控更新可写为

$$\mathbf{h}_t = \mathbf{u}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{u}_t) \odot \tilde{\mathbf{h}}_t, \quad 0 \leq \mathbf{u}_t \leq 1.$$

当 $\mathbf{u}_t$ 接近 **1** 时, 旧状态主要被保留; 当 $\mathbf{u}_t$ 接近 **0** 时, 新候选状态主要被写入. 中间取值则形成可微的 soft gate, 使模型可以逐维、逐时刻地调节累积速度.

- 这里的 $\sigma(\cdot)$ 通常指 Logistic 函数, 输出范围为 $(0, 1)$, 因而适合构造门控. 候选状态一般使用 $\tanh(\cdot)$, 因为它零中心化且输出范围为 $(-1, 1)$. 需要注意, Tanh 并不能根治梯度消失; 它只是比单纯的 Sigmoid 候选更新更适合表示正负变化.

2. 门控循环单元 GRU

- 门控循环单元 (Gated Recurrent Unit, GRU) 用更新门和重置门控制隐藏状态. 设 $\mathbf{x}_t \in \mathbb{R}^M$, $\mathbf{h}_{t-1} \in \mathbb{R}^D$, 则

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z),$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r),$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h),$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t.$$

其中 $\mathbf{z}_t, \mathbf{r}_t, \tilde{\mathbf{h}}_t, \mathbf{h}_t \in \mathbb{R}^D$.

- 更新门 $\mathbf{z}_t$ 决定旧状态保留多少. 若某一维 $z_{t,j} \approx 1$, 则 $h_{t,j}$ 主要沿用 $h_{t-1,j}$; 若 $z_{t,j} \approx 0$, 则该维主要写入候选状态 $\tilde{h}_{t,j}$. 这解释了为什么 GRU 比简单 RNN 更容易保留跨多个时间步的信息.
- 重置门 $\mathbf{r}_t$ 决定候选状态在计算时能看到多少旧历史. 若 $r_{t,j} \approx 0$, 则对应历史分量被屏蔽, 候选状态更接近只由当前输入 $\mathbf{x}_t$ 决定; 若 $r_{t,j} \approx 1$, 候选状态会充分利用上一时刻隐藏状态. 因而 GRU 可以在“继续利用旧上下文”和“根据当前输入重新开始”之间切换.
- GRU 没有单独的记忆单元, 所有信息都保存在 $\mathbf{h}_t$ 中. 这使它比 LSTM 参数更少、计算更简单, 但也意味着存储状态和输出状态没有明确分离. 在数据量有限或序列不太长时,

GRU 往往是有效基线; 在需要更细粒度控制存储与读取时, LSTM 的结构更清晰.

3. 长短期记忆网络 LSTM

- 长短期记忆网络 (Long Short-Term Memory, LSTM) 在隐藏状态之外引入细胞状态 $\mathbf{c}_t$. 一个常见形式为

$$\begin{aligned}\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c), \\ \mathbf{i}_t &= \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \\ \mathbf{f}_t &= \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f), \\ \mathbf{o}_t &= \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o), \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t).\end{aligned}$$

其中 $\mathbf{i}_t$ 是输入门, $\mathbf{f}_t$ 是遗忘门, $\mathbf{o}_t$ 是输出门.

- 细胞状态 $\mathbf{c}_t$ 是 LSTM 的主要存储通道, 隐藏状态 $\mathbf{h}_t$ 则是暴露给当前输出层和下一时刻门控计算的状态. 这一区分很重要: GRU 把存储和输出合并在一个 $\mathbf{h}_t$ 中, LSTM 则把“记住什么”和“输出什么”拆开.
- 输入门控制候选信息写入多少, 遗忘门控制旧细胞状态保留多少, 输出门控制细胞状态向隐藏状态暴露多少. 当 $\mathbf{f}_t$ 接近 1 且 $\mathbf{i}_t$ 接近 0 时, $\mathbf{c}_t$ 可以近似沿时间复制, 从而给梯度提供较稳定的传播路径; 当 $\mathbf{f}_t$ 接近 0 时, 模型可以主动擦除已经过时的信息.
- “长短期记忆”不是说 LSTM 拥有永久长期记忆. 网络参数保存的是跨训练样本积累下来的慢变化知识, 而 $\mathbf{h}_t$ 和 $\mathbf{c}_t$ 是当前序列内部的动态状态. LSTM 的名称强调的是: 相比简单 RNN 每步重写隐藏状态, 细胞状态可以把某些关键信息保留更长的时间间隔, 但它仍然是序列内的工作记忆.
- 遗忘门也不只是“增加记忆容量”. 更准确地说, 它控制信息生命周期: 重要信息可以保留, 无关或过期信息可以删除. 若没有遗忘机制, 细胞状态可能不断累积, 使旧信息长期污染后续判断; 若遗忘过强, 又会退化成短记忆模型.

4. LSTM 的常见变体

- 早期 LSTM 可以没有显式遗忘门, 细胞状态更新为

$$\mathbf{c}_t = \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

这种结构提供了加法记忆通道, 但不容易主动删除过期信息. 后来加入遗忘门后, 模型能更稳定地控制细胞状态的保留与清除.

- 一种简化变体把输入门和遗忘门耦合起来:

$$\mathbf{c}_t = (\mathbf{1} - \mathbf{i}_t) \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

这表示写入新信息越多, 保留旧信息越少. 它减少了一个门控向量, 但也限制了“同时少量写入并大量保留”这类行为.

- Peephole 连接允许门控直接观察细胞状态, 例如

$$\begin{aligned}i_t &= \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{V}_i \mathbf{c}_{t-1} + \mathbf{b}_i), \\f_t &= \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{V}_f \mathbf{c}_{t-1} + \mathbf{b}_f), \\o_t &= \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{V}_o \mathbf{c}_t + \mathbf{b}_o).\end{aligned}$$

这类连接在需要精细计时的任务中可能有帮助, 因为门不只依赖输入和上一隐藏状态, 还可以根据细胞内部已有内容做决定. 但它增加参数和实现复杂度, 因而不是所有工程实现的默认选择.

5. RNN 的能力边界与并行性问题

- 从状态递推角度看, $\mathbf{h}_t$ 或 $\mathbf{c}_t$ 可以隐含之前所有时刻输入的信息. 但“可以隐含”不等于“能够无损记住”: 状态维度有限, 梯度传播会衰减或爆炸, 训练数据也未必足以教会模型保留正确内容. 因此, 标准 RNN 的有效记忆时间通常明显短于理论上可展开的序列长度.
- 从表达能力角度看, 前馈神经网络的通用近似定理讨论的是固定维度函数逼近; 循环网络则更像学习一个随时间执行的状态程序或非线性动力系统. 在理想化条件下, 某些循环网络具有图灵完备性, 可以模拟任意可计算过程. 但这只是表达能力的存在性结论, 并不保证有限数据和梯度下降一定能找到这样的参数.
- 若任务需要保存和读取大量外部信息, 可以在 RNN 外部加入记忆模块或注意力机制. 注意力不是把所有历史都塞进一个固定维度状态, 而是在需要从一组历史表示中读取相关位置, 因而能缓解单一状态向量的信息瓶颈.
- RNN 的另一个局限是并行能力. 由于 $\mathbf{h}_t$ 依赖 $\mathbf{h}_{t-1}$, 同一条序列上的时间步通常必须按顺序计算; CNN 和全连接网络则更容易在空间位置或样本维度上并行. 工程上可以并行不同样本、并行每个时间步内部的矩阵乘法, 但时间维度上的递推依赖仍然存在.
- 提升并行性的常见思路包括: 用一维卷积或时序卷积网络在固定感受野内并行处理局部上下文, 用 Transformer 自注意力让所有位置直接互相读取, 或用可并行扫描的状态空间模型近似长序列递推. 这些方法与门控 RNN 的目标相通, 都是在寻找更稳定、更高效的长程信息传递方式.

6. 训练策略与实际局限

- BPTT 的完整展开长度为 T , 计算和存储开销都随序列长度增长. 实际训练中常用截断 BPTT, 即只向前反传固定步数. 这样可以控制计算成本, 但也会限制模型直接学习更远依赖的能力.
- 标准 RNN、GRU 和 LSTM 都是在时间维度上反复应用同一状态更新函数. 门控机制改善了梯度路径和记忆控制, 但不会完全消除梯度爆炸、过拟合、训练慢和长序列信息丢失等问题. 因此, 梯度截断、合适初始化、层归一化、dropout 和残差连接仍然常与门控 RNN 一起使用.
- 选择 GRU、LSTM 还是其他序列模型, 取决于任务长度、数据规模、延迟要求和可解释性需求. 若序列较短或工程资源有限, GRU 常提供简单稳健的基线; 若需要明确区分存储与输出, LSTM 更合适; 若需要大规模并行训练和直接访问长上下文, 注意力模型通常更有优势.

6.1 现代 RNN

Transformer 取代传统 RNN 成为大语言模型主流以后, RNN 并没有完全退出序列建模. 近年的 RWKV 和 Mamba 等工作重新利用“逐步更新状态”的思想, 但它们并不是简单回到 vanilla RNN, 而是把循环状态、门控、线性注意力或状态空间模型与现代硬件上的并行训练结合起来.

1. 先用直觉理解 Transformer 的限制

- Transformer 可以先粗略理解为一种“让序列中每个位置都看一眼其他位置”的模型². 在处理一句话时, 每个 token 都会生成查询、键和值, 再根据查询和键的相似度决定从哪些位置读取信息. 这使它很擅长捕捉远距离依赖: 句首的信息可以在一层中直接影响句尾表示, 不必像 RNN 那样一步步传递.
- 这个机制的代价是成对比较. 若序列长度为 T , 自注意力大致需要计算 $T \times T$ 个位置关系, 因而时间和显存开销会随 T 二次增长. 当上下文从几千 token 扩到几十万 token 时, 注意力矩阵会迅速变成主要负担.
- 另一个限制来自上下文窗口. 实际 Transformer 通常只能在一次前向计算中接收有限长度的 token, 这与训练长度、位置编码、显存和推理开销都有关. RoPE、滑动窗口注意力和检索增强等方法可以缓解这个问题, 但它们仍然是在“全局两两比较太贵”这一约束下做工程折中.

2. RWKV 模型: 把注意力写成可循环更新的形式

- RWKV 的全称是 Receptance Weighted Key Value³. 它的核心想法是把一类线性注意力改写成既可以并行训练、又可以像 RNN 一样逐 token 推理的结构. 训练时, 模型可以像 Transformer 那样利用整段序列做并行计算; 推理时, 它只维护一个随时间更新的状态, 因而每生成一个新 token 不需要重新保存和访问完整注意力矩阵.
- 从本节课角度看, RWKV 体现的是“RNN 的状态递推并没有过时, 过时的是早期简单递推在表达能力、并行训练和规模化上的不足”. 它把注意力中的 key-value 汇聚压缩进循环状态, 因而推理复杂度可以随序列长度线性增长. 但这也意味着信息仍要经过有限状态瓶颈, 对需要精确回看很早细节的任务, 它未必等价于完整自注意力.

3. Mamba: 让状态空间模型学会选择性记忆

- Mamba 属于选择性状态空间模型 (selective state space model)⁴. 状态空间模型本来就可以写成递推形式: 当前状态接收当前输入并产生当前输出. Mamba 的关键改进是让状态更新参数依赖当前输入, 使模型能够根据当前 token 选择性地传播、写入或遗忘信息.
- 这和 LSTM/GRU 的门控思想有直接联系: 二者都不是无条件保存全部历史, 而是学习“什么该留、什么该丢”. 区别在于, Mamba 使用结构化状态空间和硬件友好的 selective scan 算法, 试图同时获得长序列上的线性扩展能力、较大的有效状态容量和 GPU 上可接受的并行效率. 因此, 它可以看作把本节的门控记忆思想推进到现代长序列建模中的一个代表.

4. 现代 RNN 的复兴应如何理解

²Ashish Vaswani et al., *Attention Is All You Need*, arXiv:1706.03762, 2017, <https://arxiv.org/abs/1706.03762>.

³Bo Peng 等, *RWKV: Reinventing RNNs for the Transformer Era*, arXiv:2305.13048, 2023, <https://arxiv.org/abs/2305.13048>.

⁴Albert Gu and Tri Dao, *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*, arXiv:2312.00752, 2023, <https://arxiv.org/abs/2312.00752>.

- 现代 RNN 的目标不是证明 Transformer 不重要, 而是在 Transformer 的强表达能力和 RNN 的线性推理效率之间寻找折中. Transformer 像是把一段上下文摊开成一张关系表, 每个位置都能直接查其他位置; RNN 类模型则像是边读边更新一本压缩笔记, 每来一个新 token 就修改当前状态.
- 压缩笔记的好处是省空间、适合流式输入, 理论上可以一直读下去; 缺点是笔记容量有限, 写错或丢掉的信息后面很难恢复. 因此, RWKV、Mamba 等模型真正要解决的核心问题, 仍然是本节一直讨论的三个主题: 如何保存长程依赖, 如何控制记忆容量, 以及如何让序列模型在现代硬件上高效训练和推理.

问题思考:

- GRU 的更新门和重置门分别控制什么? 当它们接近 0 或 1 时, 状态更新会发生什么变化?
- 为什么说 LSTM 的 c_t 是序列内的工作记忆, 而网络参数才是跨样本学习得到的慢变化知识?
- RWKV 和 Mamba 都强调线性时间序列建模, 但它们为什么仍然不能简单等同于“无限无损记忆”?

(注: 详细定义和更多内容请参考课程教材第 6 章, 参考课程: Stanford CS231N | Spring 2025)